NumPy Array Attributes

In NumPy, attributes are properties of array objects that provide important information about the arrays and their data. These attributes are used to access various details regarding the structure and configuration of the arrays without modifying them.

- **NumPy Shape Attribute**

The NumPy shape attribute provides the dimensions of the array. It returns a tuple representing the size of the array along each dimension. It can also be used to resize the array.

Example 1

```
import numpy as np

a = np.array([[1,2,3],[4,5,6]])

print (a.shape)
```

OutPut

```
(2, 3)
```

Example 2 resizing an array using the shape attribute

```
import numpy as np
```

```
a = np.array([[1,2,3],[4,5,6]])

a.shape = (3,2)

print (a)
```

Output

```
[[1, 2]

 [3, 4]

 [5, 6]]
```

Example 3 reshape() function to resize an array

```
import numpy as np

a = np.array([[1,2,3],[4,5,6]])

b = a.reshape(3,2)

print (b)
```

Output

```
[[1, 2]

 [3, 4]

 [5, 6]]
```

- **NumPy Dimensions Attribute**

The ndim attribute returns the number of dimensions (axes) of the array.

In NumPy, the dimension of an array is known as its rank. Each axis in a NumPy array corresponds to a dimension. The number of axes (dimensions) is referred to as the array's rank.

Arrays can be of any dimension, from one-dimensional (1D) arrays (also known as vectors) to multi-dimensional arrays like 2D arrays (matrices) or even higher-dimensional arrays.

Example 1

```
import numpy as np

a = np.arange(24)

print (a)
```

Output

```
[0 1  2  3  4  5  6  7  8  9  10  11  12  13  14  15  16 17 18 19 20 21 22 23]
```

Example 2

Here, we are creating a one-dimensional NumPy array a with "24" elements using the arange() function and then reshaping it into a three-dimensional array "b" with the shape provided, resulting in a 3D array −

```
import numpy as np

a = np.arange(24)

a.ndim
```

```
# Now reshape it

b = a.reshape(2,4,3)

print (b)

# b is having three dimensions
```

Output

```
[[[ 0,  1,  2]

  [ 3,  4,  5]

  [ 6,  7,  8]

  [ 9, 10, 11]]

  [[12, 13, 14]

   [15, 16, 17]

   [18, 19, 20]

   [21, 22, 23]]]
```

- **NumPy Size Attribute**

The size attribute returns the total number of elements in the array. In NumPy, the size of an array refers to the total number of elements contained within the array.

For a one-dimensional array, the size is simply the number of elements.

For a two-dimensional array, the size is the product of the number of rows and columns.

For a three-dimensional array, the size is the product of the sizes of all three dimensions.

Example

```
import numpy as np

# Creating a 3D array

array_3d = np.array([[[1, 2, 3], [4, 5, 6]],

                     [[7, 8, 9], [10, 11, 12]]])

print("3D Array:\n", array_3d)

print("Size of the array:", array_3d.size)
```

Output

```
3D Array:

[[[ 1  2  3]

  [ 4  5  6]]



 [[ 7  8  9]

  [10 11 12]]]

Size of the array: 12
```

- **NumPy Data Type Attribute**

The dtype attribute describes the data type of the elements in the array. NumPy supports a wide range of data types, including integers, floats, complex numbers, booleans, and more. Each data type is represented by a dtype object. The "dtype" not only specifies the type of the data but also its size and byte order.

Example

```
import numpy as np

int_array = np.array([1, 2, 3], dtype=np.int32)

print("Data type of int_array:", int_array.dtype)


float_array = np.array([1.1, 2.2, 3.3], dtype=np.float64)

print("Data type of float_array:", float_array.dtype)
```

- **NumPy Itemsize Attribute**

The itemsize attribute returns the the length of each element of array in bytes.

The item size is determined by the data type (dtype) of the array. Different data types require different amounts of memory. For example, an int32 type requires "4" bytes per element, while a float64 type requires "8" bytes per element.

Example 1

```
# dtype of array is int8 (1 byte)

import numpy as np

x = np.array([1,2,3,4,5], dtype = np.int8)

print (x.itemsize)
```

Output

```
1
```

Example 2

```
import numpy as np

x = np.array([1,2,3,4,5], dtype = np.float32)

print (x.itemsize)
```

Output

```
4
```

NumPy Arithmetic Operations

NumPy makes performing arithmetic operations on arrays simple and easy. With NumPy, you can add, subtract, multiply, and divide entire arrays element-wise, meaning that each element in one array is operated on by the corresponding element in another array.

When performing arithmetic operations with arrays of different shapes, NumPy uses a feature called broadcasting. It automatically adjusts the shapes of the arrays so that the operation can be performed, extending the smaller array across the larger one as needed.

- **NumPy Array Addition**

Addition in NumPy is performed element-wise. When two arrays of the same shape are added, corresponding elements are summed together.

```
import numpy as np


# Creating two arrays

a = np.array([1, 2, 3])

b = np.array([4, 5, 6])

# Adding arrays

result = a + b
```

```
print(result)
```

Output

```
[5 7 9]
```

- **NumPy Array Subtraction**

Subtraction in NumPy is also element-wise. Subtracting two arrays with the same shape returns an array where each element is the difference of the corresponding elements in the input arrays −

```
import numpy as np

# Creating two arrays

a = np.array([10, 20, 30])

b = np.array([1, 2, 3])

# Subtracting arrays

result = a - b

print(result)
```

Output

```
[ 9 18 27]
```

- **NumPy Array Multiplication**

Element-wise multiplication is performed using the * operator in NumPy. When multiplying arrays, each element of the first array is multiplied by the corresponding element in the second array −

```
import numpy as np

# Creating two arrays

a = np.array([1, 2, 3])

b = np.array([4, 5, 6])

# Multiplying arrays

result = a * b

print(result)
```

Output

```
[ 4 10 18]
```

- **NumPy Array Division**

Division is performed element-wise using the / operator in NumPy. It results in an array where each element is the quotient of the corresponding elements in the input arrays −

```
import numpy as np
```

```
# Creating two arrays

a = np.array([10, 20, 30])

b = np.array([1, 2, 5])

# Dividing arrays

result = a / b

print(result)
```

Output

```
[10. 10.  6.]
```

- **NumPy Array Power Operation**

```
import numpy as np

# Creating two arrays

a = np.array([2, 3, 4])

b = np.array([1, 2, 3])

# Applying power operation

result = a ** b

print(result)
```

Output

```
[ 2  9 64]
```

- **NumPy Modulo Operation**

```
import numpy as np


# Creating two arrays

a = np.array([10, 20, 30])

b = np.array([3, 7, 8])

# Applying modulo operation or use print (np.mod(a,b))

result = a % b

print(result)
```

Output

```
[1 6 6]
```

- **NumPy Floor Division**

```
import numpy as np

# Creating two arrays

a = np.array([10, 20, 30])

b = np.array([3, 7, 8])

# Applying floor division

result = a // b
```

```
print(result)
```

Output

```
[3 2 3]
```

NumPy Sum Operation

The NumPy sum operation calculates the sum of array elements over a specified axis or over the entire array if no axis is specified. −

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])

result = np.sum(a)

print(result)

# Summing along axis 0 (columns)

result_axis0 = np.sum(a, axis=0)

print(result_axis0)

# Summing along axis 1 (rows)

result_axis1 = np.sum(a, axis=1)

print(result_axis1)
```

Output

```
21
```

[5 7 9]

[6 15]

NumPy Mean Operation

The NumPy mean operation calculates the average (arithmetic mean) of array elements over a specified axis or over the entire array −

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])

# Mean of elements

result = np.mean(a)

print(result)

# Mean along axis 0 (columns)

result_axis0 = np.mean(a, axis=0)

print(result_axis0)

# Mean along axis 1 (rows)

result_axis1 = np.mean(a, axis=1)

print(result_axis1)
```

Output

3.5

[2.5 3.5 4.5]

[2. 5.]

NumPy Array Operations with Complex Numbers

The following functions are used to perform operations on array with complex numbers.

numpy.real(): Returns the real part of the complex data type argument.

numpy.imag(): Returns the imaginary part of the complex data type argument.

numpy.conj(): Returns the complex conjugate, which is obtained by changing the sign of the imaginary part.

numpy.angle(): Returns the angle of the complex argument. The function has degree parameter. If true, the angle in the degree is returned, otherwise the angle is in radians.

Example

```
import numpy as np

a = np.array([-5.6j, 0.2j, 11. , 1+1j])


print ('Our array is:')
```

```
print (a)

print ('\nApplying real() function:')

print (np.real(a))

print ('\nApplying imag() function:')

print (np.imag(a))

print ('\nApplying conj() function:')

print (np.conj(a))

print ('\nApplying angle() function:')

print (np.angle(a))

print ('\nApplying angle() function again (result in degrees)')

print (np.angle(a, deg = True))
```

Output

```
Applying real() function:

[ 0. 0. 11. 1.]

Applying imag() function:

[-5.6 0.2 0. 1. ]

Applying conj() function:

[ 0.+5.6j 0.-0.2j 11.-0.j 1.-1.j ]

Applying angle() function:
```

[-1.57079633 1.57079633 0. 0.78539816]

Applying angle() function again (result in degrees)

[-90. 90. 0. 45.]

Basic Array Operations

NumPy arrays also routines for operations like additions, subtraction and indexing for efficient data manipulation −

Operation	Description
numpy.add()	Enables efficient element-wise addition for data manipulation.
numpy.subtract()	Computes element-wise differences between two arrays efficiently.
numpy.multiply()	Computes element-wise products between two arrays efficiently.
numpy.divide()	Performs element-wise division, requiring non-zero, same-shaped arrays.
numpy.power()	Raises elements of one array to another's element-wise.
numpy.mod()	Computes element-wise remainders of division between two arrays.

numpy.remainder()	Computes element-wise remainders of division between two arrays.
numpy.divmod()	Returns a tuple with quotient and remainder of division.
numpy.abs()	Returns the positive absolute value of numbers.
numpy.absolute()	Computes the absolute value of each array element efficiently.
numpy.fabs()	Returns the positive magnitudes.
numpy.sign()	Returns an element-wise indication of the sign of a number.
numpy.conj()	Returns the complex conjugate, element-wise.
numpy.exp()	Calculates the exponential of all elements in the input array.
numpy.sqrt()	Returns the non-negative square-root of an array, element-wise.
numpy.square()	Returns the element-wise square of the input.